Isabel Silva

CPSC 362, Assignment 2

Introduction:

The Problem – Due to covid restrictions our customers (restaurants and food services) are limited to 25% max capacity for indoor dining, remote services have increased, and there is little to no social gatherings that require food services. Due to these events we are unable to attract new customers and current customers cannot sustain themselves or need help. Therefore, we must develop a system that provides logistics for restaurants in a post-covid world.

Solution – Create a web/mobile application that will hold a collective of restaurant's logistics. For our current customers we will add their logistics to this new web/mobile application as well as add new features 1-3 (detailed bellow in application description) that will entice consumers to choose our customer's services.

Intended Goals

Build on current data to limit time/cost of production that will eventually minimize/regulate customer's fees. Low costs will entice our customers with us as well as potentially signing up new ones.

Enhance our new features so that our customer's consumer's (users) will feel comfortable enough during these troubling times to use our customer's services

Intended Users

Admin – complete access to the application and databases for regular maintenance, security, customer/user inquires, customer pages build. This are the software developers and engineer of the application.

Customer – the restaurants and food services that contract our services

User – the intended consumer for the customer

Application Service Requirements

The following are a list of services for this application.

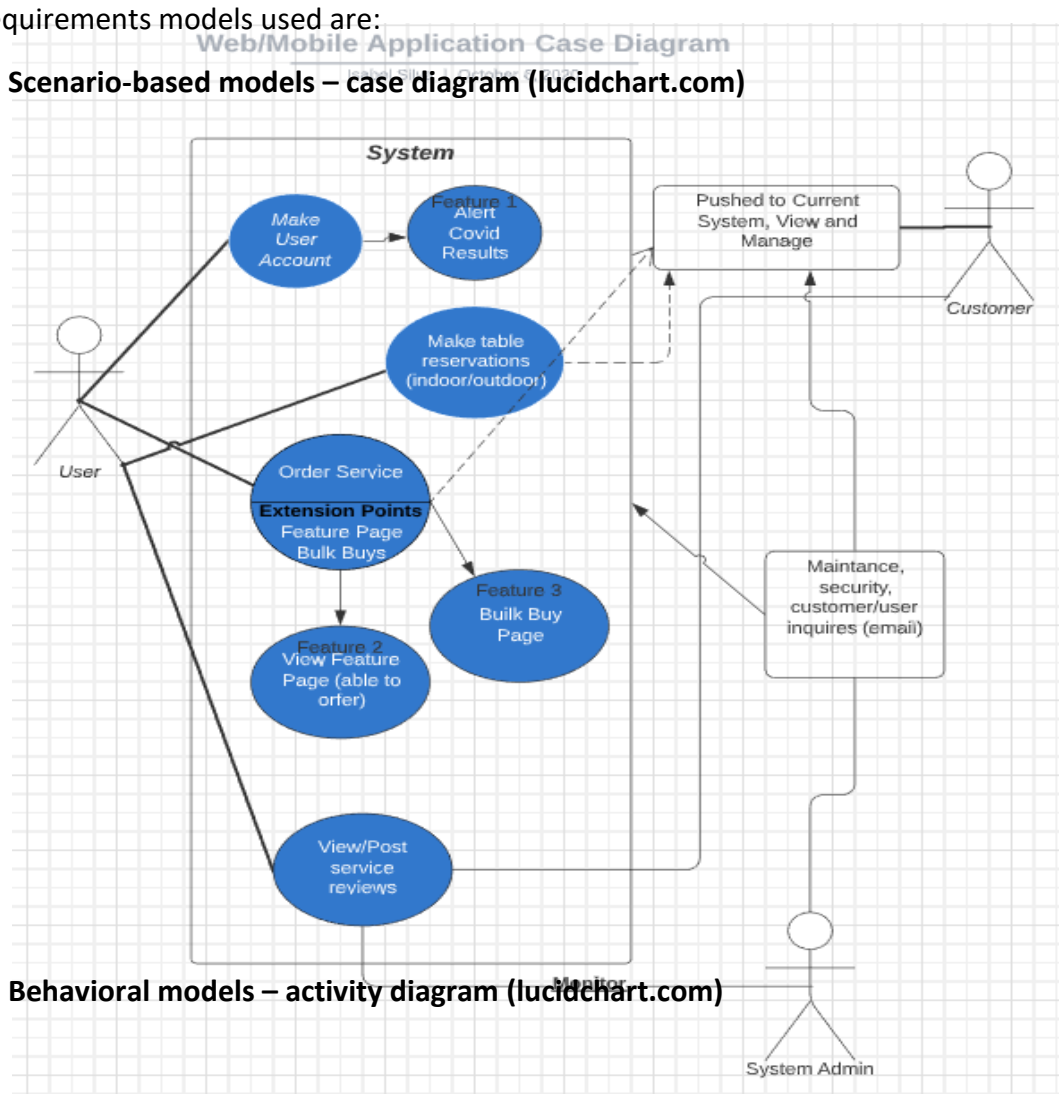
- User create user account
- User Order food for pickup/curbside from menu (order-ahead capable). Customer may view and manage food ordering.
- User Check outdoor/indoor seating (capacity/availability if applicable). Customers can update this information via integrated system that has access to customer's database.

- User can check and make Table/room/services reservations and see availability. Customers can view and manage this information.
- User can view/post reviews along with images; customers can manage review's (reply) and flag for inappropriate conduct. Admin can view/manage inappropriate conduct.
- **Feature 1**: Covid Tracking - keep track of user that have reported being diagnosed with Covid marking the day and time of visit (user must include full name of themselves and party when using dining services)
- **Feature 2**: Feature pages for customers who have wine or special items that they normally will sell while dining, however unable to because of Covid restrictions. The user can buy the item as a whole (example bottle sales) or parts of a special item's logistics are handled by customers. Customer can update and manage the feature pages.
- **Feature 3**: User able to Bulk buy, for people looking to stock up days of food (can be prepped for oven ready at home cooking) or something similar. This can be for food services and/or restaurants. Customer can manage and update this service.

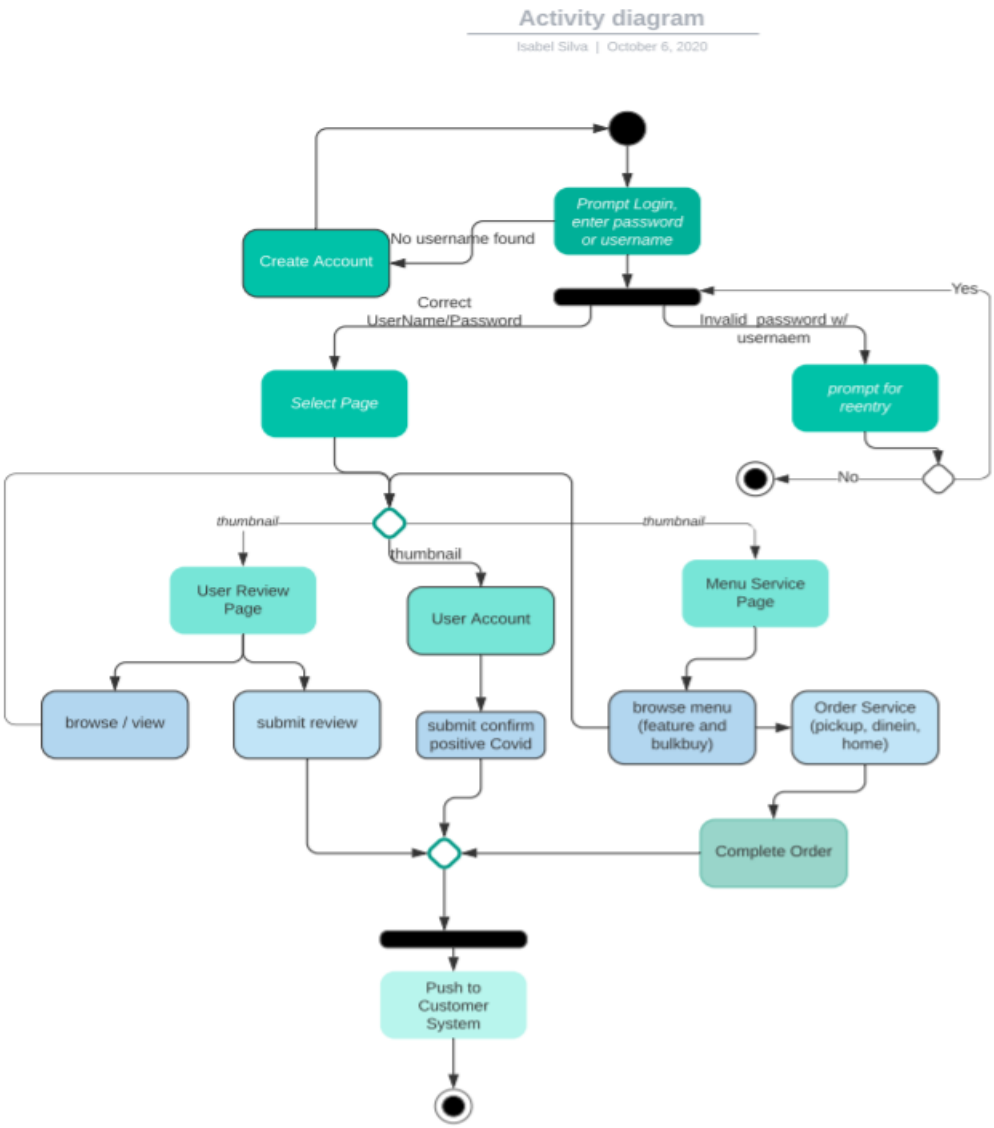
Requirement Models

Two requirements models used are:

1. **Scenario-based models – case diagram (lucidchart.com)**



2. **Behavioral models – activity diagram (lucidchart.com)**



Architectural Models:

Three architectural models[1] to evaluate are:

1. **Data-centered architectures** is an architectural style that a database/repository is located at the center of the architecture and can be accessed on a regular basis by other components that have the functionality to modify the database. Sometimes access to the database is passive, meaning that the client software can accesses the data but no changes are made. A variation of passive and non-passive transforms the database repository into a blackboard where the blackboard sends

[1] Definitions were found in textbook, __ chapter 10

notifications to the client software when data of interest to the client changes (in our application this will be when items become out of stock or unavailable, ect.).

   a. **Pro:** Promotes integrability, existing components execute independently therefore changes can be made and added without worrying about other clients. Incorporates a user interface apart from the main database.
   b. **Con:** limited to remote use, our system needs to be able to access web for mobile devices.

2. **Call-and-Return Architecture** has two substyles the main program/subprogram architectures, and the remote procedure call architectures. The main program/subprogram is a a classic model where the main program invokes other programs. The remote procedure call's main program/subprogram are distributed across multiple computers on the same network in a remote setting.
   a. **Pro:** enables the system to be modified and scaled easily.
   b. **Con:** lacks a user interface, mostly logistics software based and remote.
3. **Model–View–Controller (MVC)** model contains the application-specific content and processing logic, so essentially the functionality of the application. View contains interface specific functions, so essentially the front-end user interface (html code). Controller manages user requests, changes states for the model and selects the view responses, it coordinates the data flow between the controller and view.
   a. **Pro**: Often used for web-based development. Model data can access stored data (SQL databases). View is constantly requesting for updates of model data therefore data is current.
   b. **Con**: Model data must be formatted and organized before being transmitted to the view.

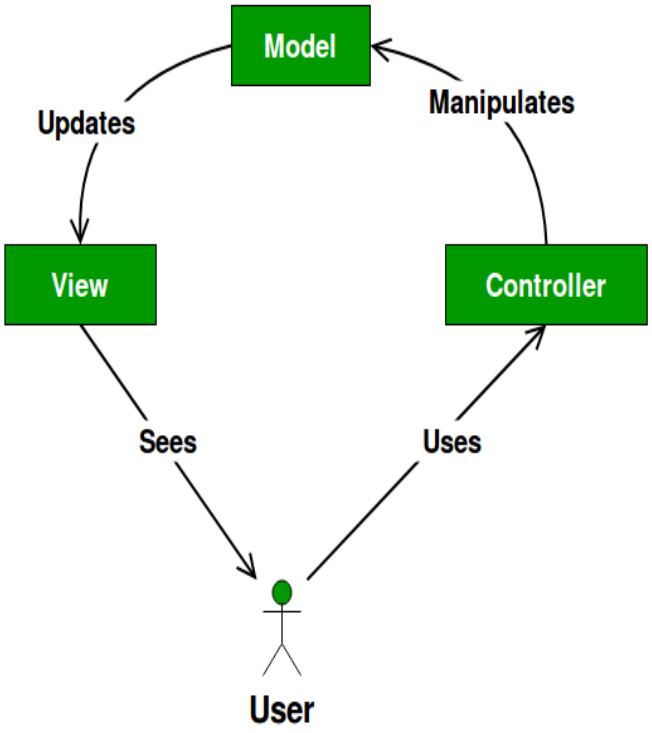
The one I will choose for my web/mobile application is the model-view-controller (MVC) architecture because our web/mobile application requires web access as well as access to our customer's database (logistics).

Isabel Silva
CPSC 362
Assigment 3

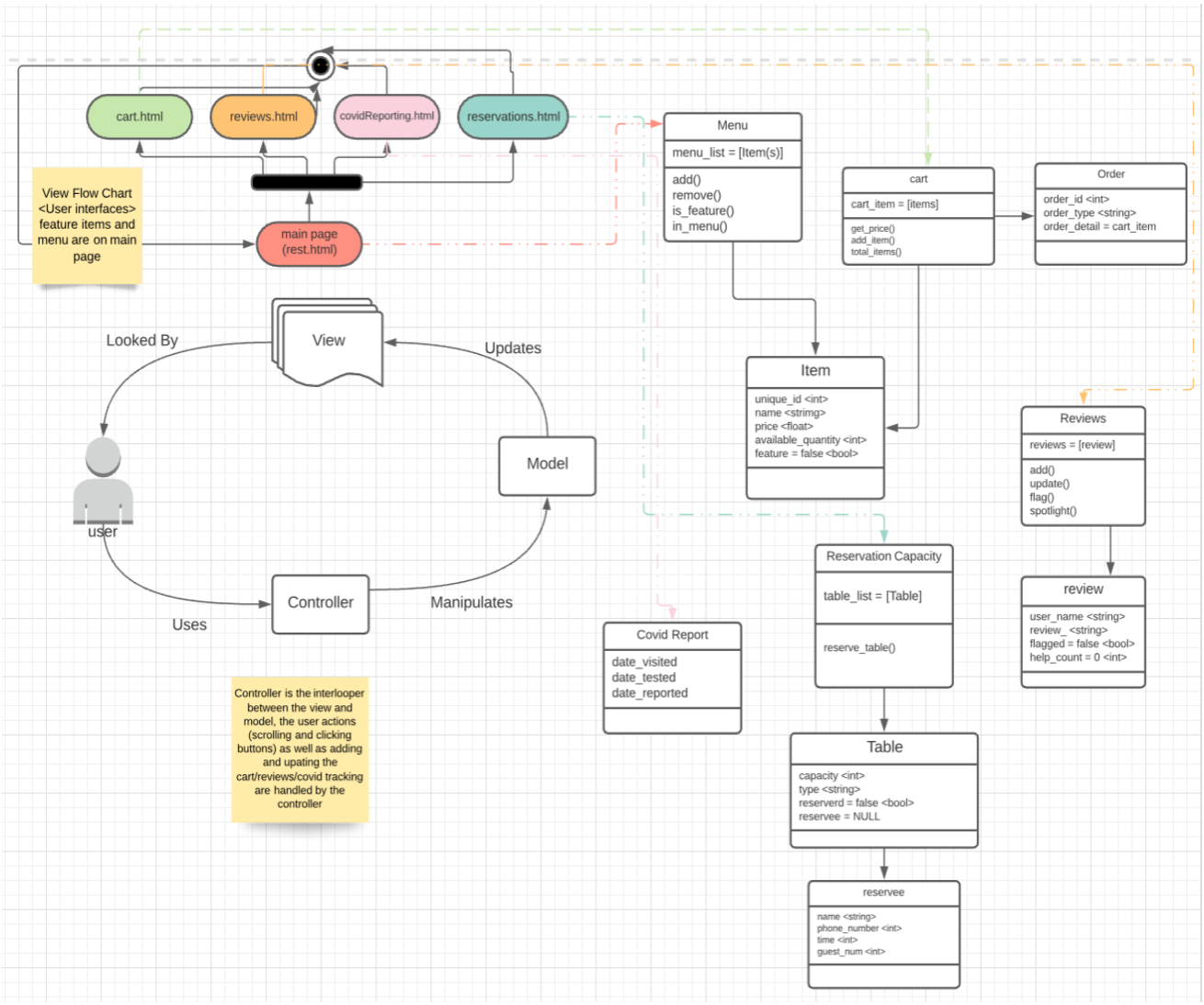
Link to webpage - https://isabel2296.github.io/restFiles/rest.html

Model-View-Controller (MVC) is the architecture chosen for this model and consists of the model, view, and controller. Below is a description and image of an MVC (*MVC Design Pattern*).

- Model – contains the application-specific content and processing logic, so essentially the functionality of the application. In this case our model will be constructed with JavaScript.
- View – contains the interface specific functions, essentially the front-end user interface (html code). In this case the view will be our html front end user interface.
- Controller – manages user requests, changes states from the model and selects the view responses, it coordinates the data flow between the controller and view.



For the webpage I got a template from a company that provides free fully responsive customizable webpages. The following is the design for the view, model, and controller (done using lucid.app).



Design Description:

- The View Flow chart is how the view will be handled. The view will consist of html and css (mainly already provided by the template but heavily modified to fit this assignment).
    - The main page will be the first view for the user and there the user may navigate through the pages. Within the main page the user can scroll through the featured items + menu items and place those items in the cart.
    - The main page also holds index's that scroll down with the user, those indices index to the appreciate destination.
        1. Reservation index: showcase reservation index and displays a check/make reservation button
            - Button leads to reservation page
        2. Featured items index: showcase featured items were user can add item to cart – user stays on main page
        3. Bulk sales index: showcase bulk sales – user stays on main page
        4. Reviews index: showcase spotlight reviews and a more reviews button

        - Button leads to review page
    5. Cart – leads to cart page
    6. Covid Reporting index: showcase number of reported
        - Button leads to report page.

- The Model Class Diagram shows the classes within the model.js file that will handle holding the backend information (database) where the controller will manipulate and change when necessary
- Controller handles all actions as well as updating the model so the model can update the view

Citations

MVC Design Pattern. 8 Feb. 2018, www.geeksforgeeks.org/mvc-design-pattern/.

Isabel Silva

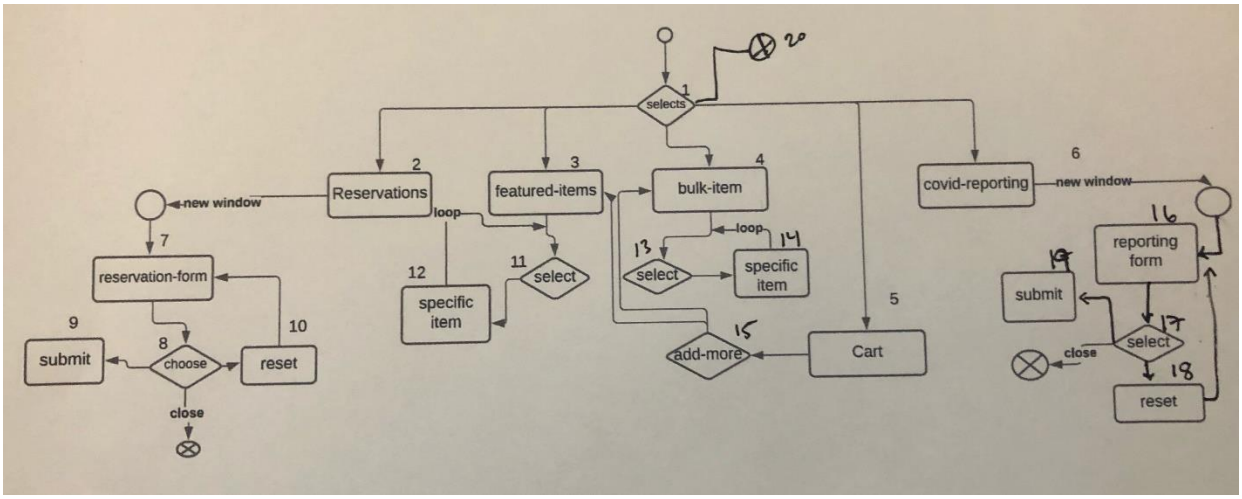
Assignment 4 Testing

Link to webpage: https://isabel2296.github.io/restFiles/rest.html

Link to video: https://www.youtube.com/watch?v=9gQJFoUazTw&feature=youtu.be

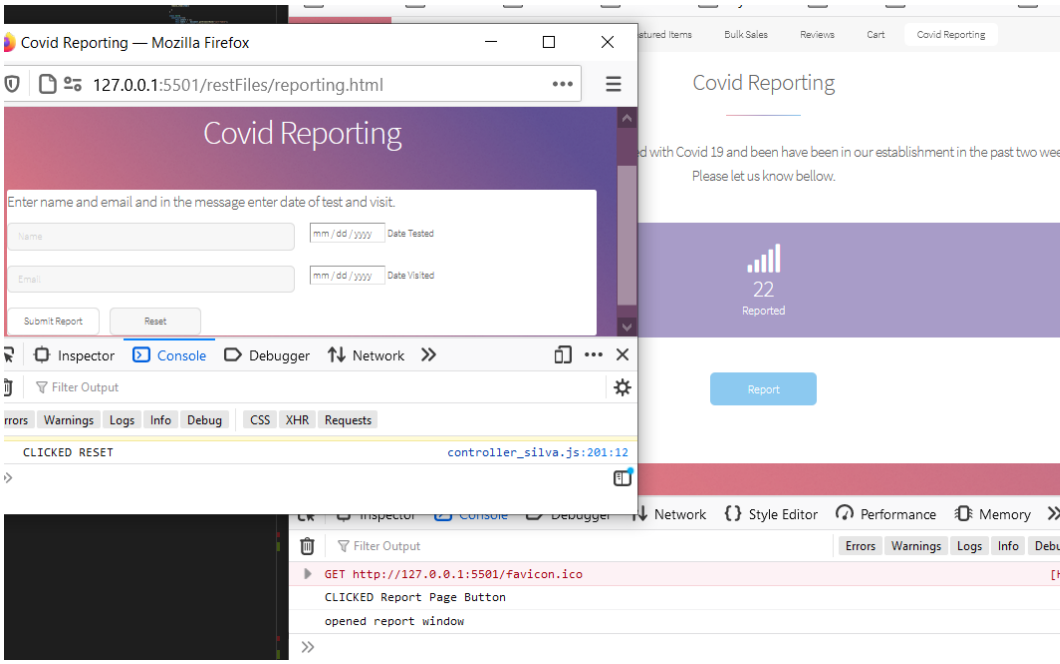
The following are images of when I tested the functionalities of my features. Recall that the features I worked on was being able to select to purchase featured items and bulk buy, as well as report covid results.

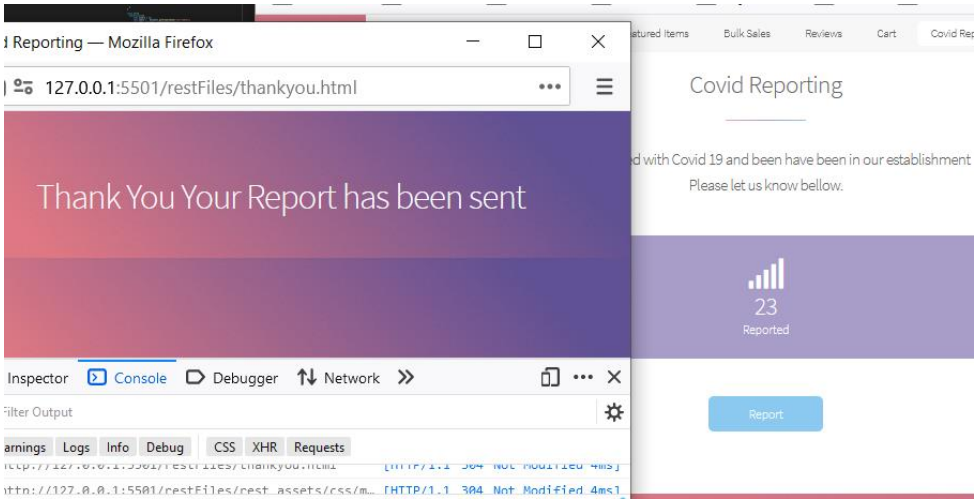
White Box Testing: Flow Chart



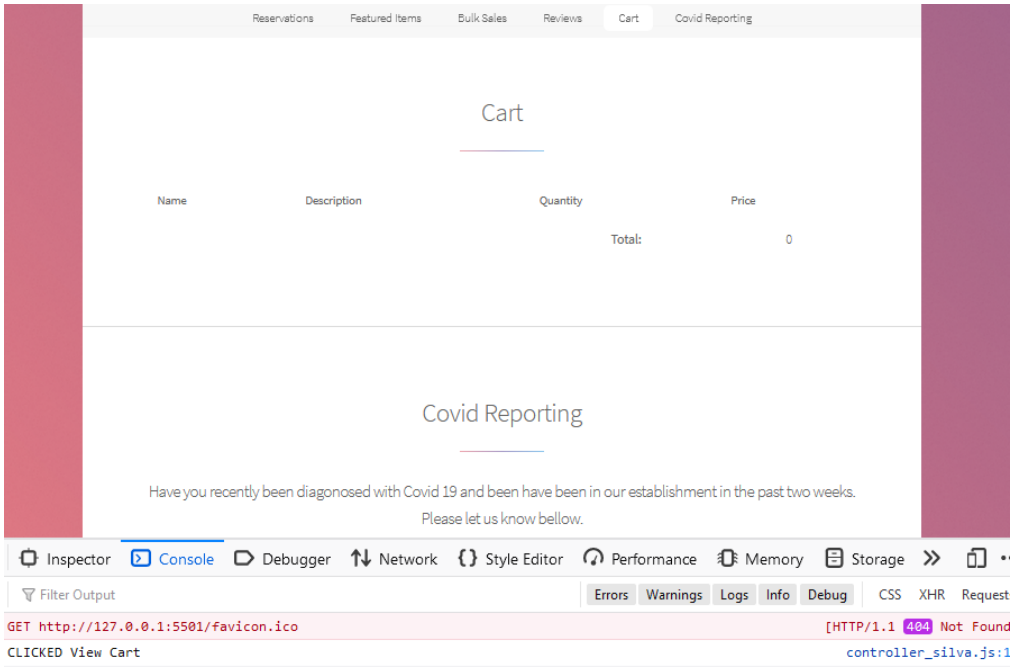
Black Box Testing: Interface Testing

- Here I am checking that the website works with the controller.js file, so if the user requests to view cart, add items, report COVID-19, or make reservations works well with the controller.js .
    - Report Covid Testing
        - From the main page the user can scroll down or click on Covid Reporting where it will go to the covid reporting section
        - For testing I created print statements when each action is called to make sure each action is working cohesion with the controller.js. Testing here is the report page pop up and reset clicked.

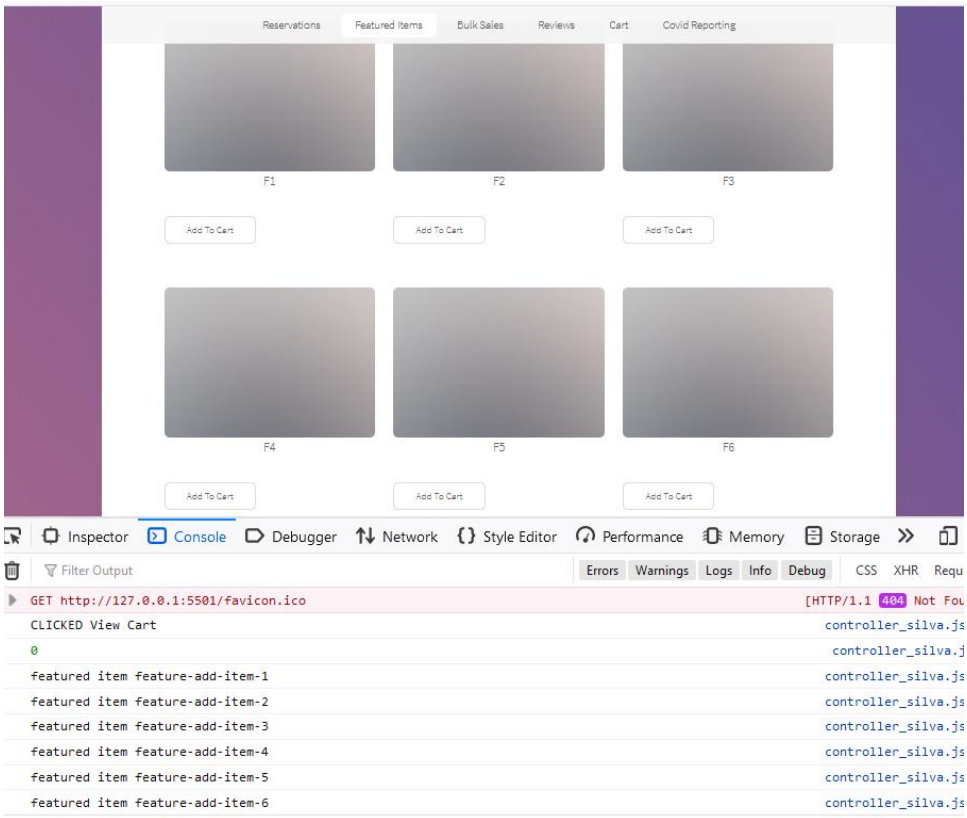
- 

  - Here the submit button will pop up a thank you page and will increment the reported from 22 to 23 on the main page
- 

- Add To cart both featured items and bulk item
  - Currently have an empty cart



- Adding all featured items once testing all buttons work

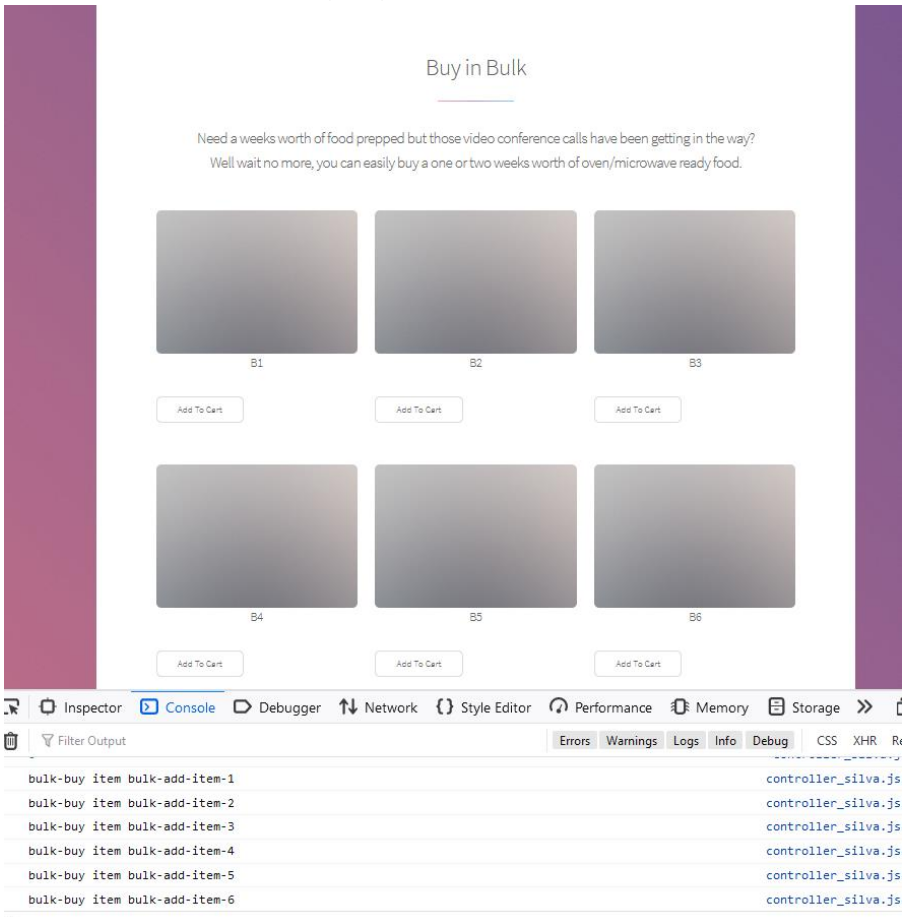


- Cart does reflect what was added (6 items)

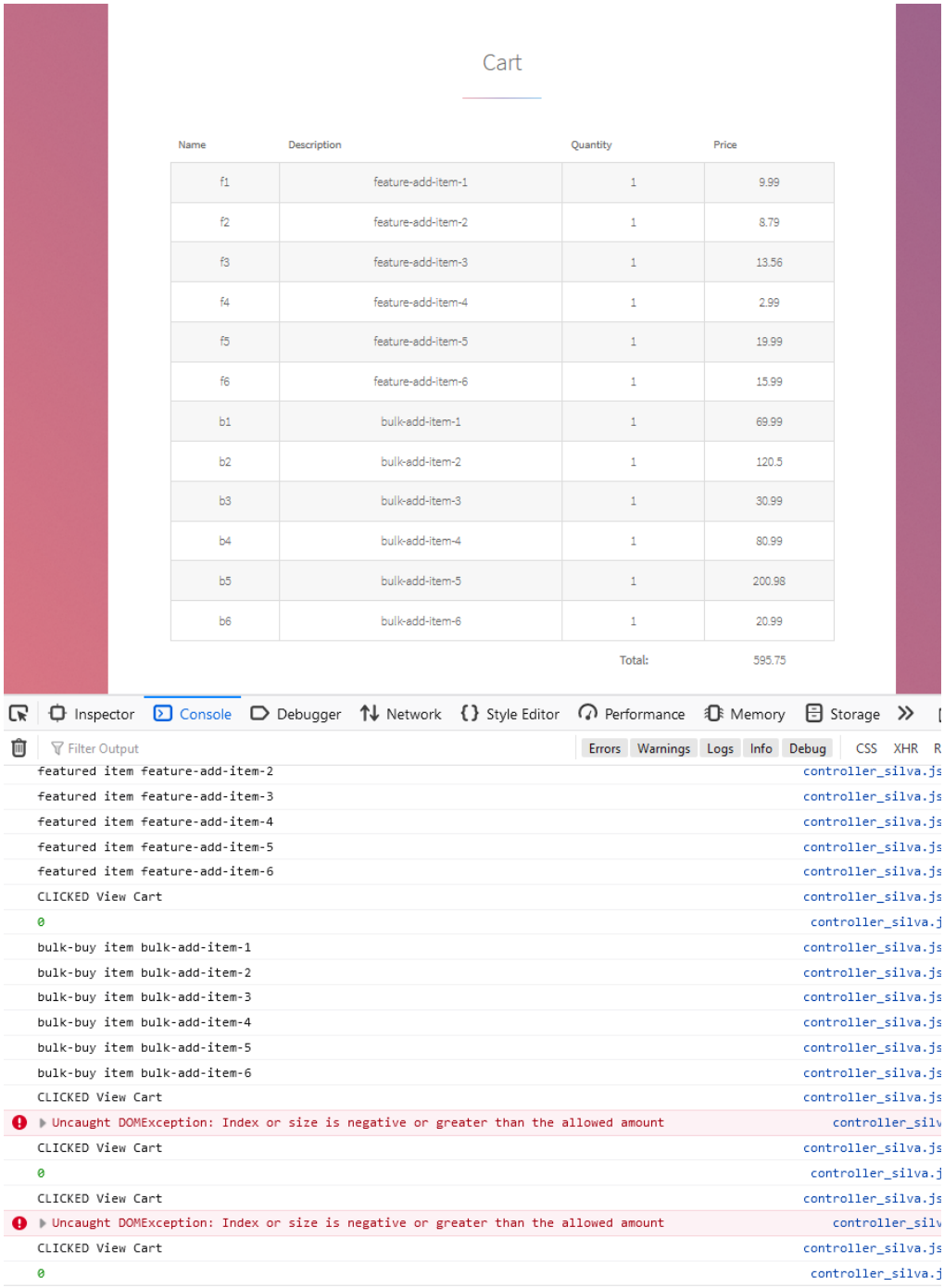
Cart

Name	Description	Quantity	Price
f1	feature-add-item-1	1	9.99
f2	feature-add-item-2	1	8.79
f3	feature-add-item-3	1	13.56
f4	feature-add-item-4	1	2.99
f5	feature-add-item-5	1	19.99
f6	feature-add-item-6	1	15.99
		Total:	71.31

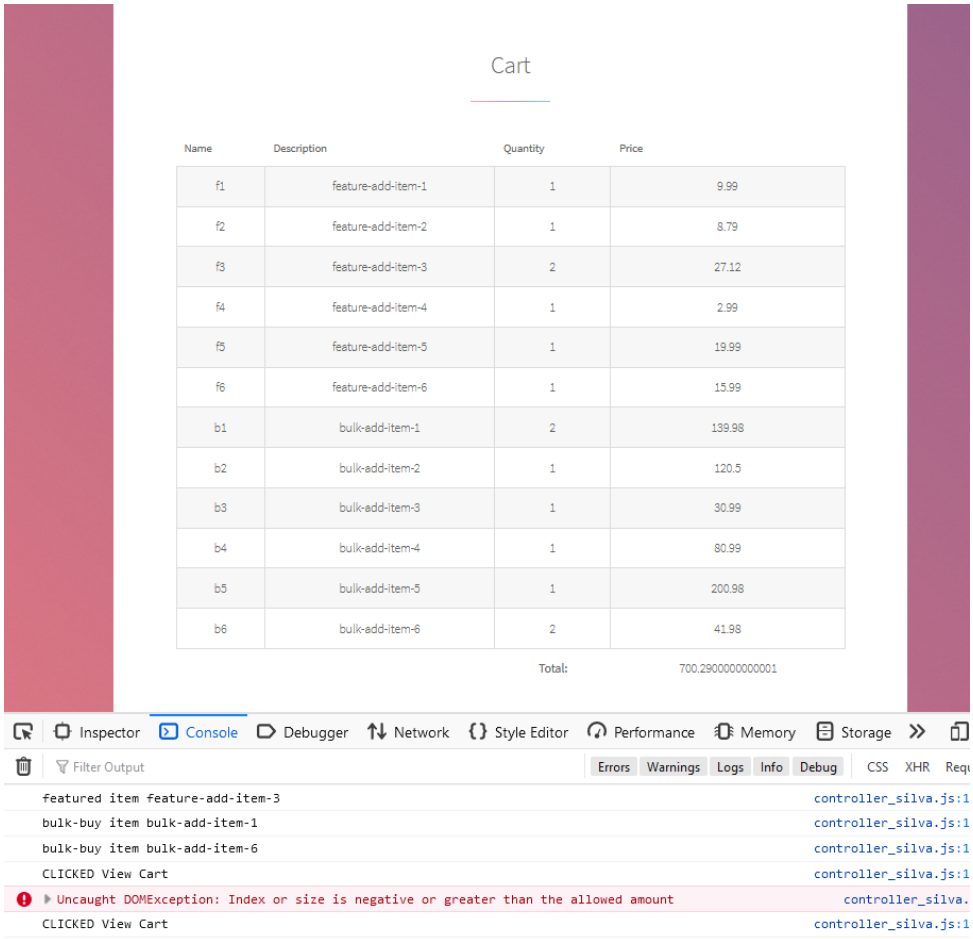
-
- Adding all the bulk items next. Testing if the buttons worked with the controller and were added to the list (cart)

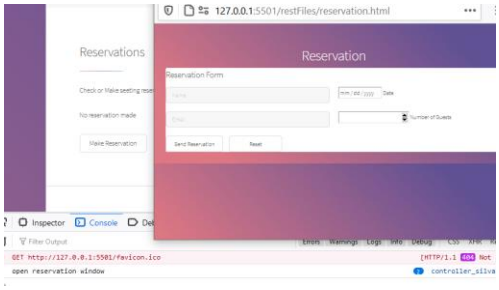


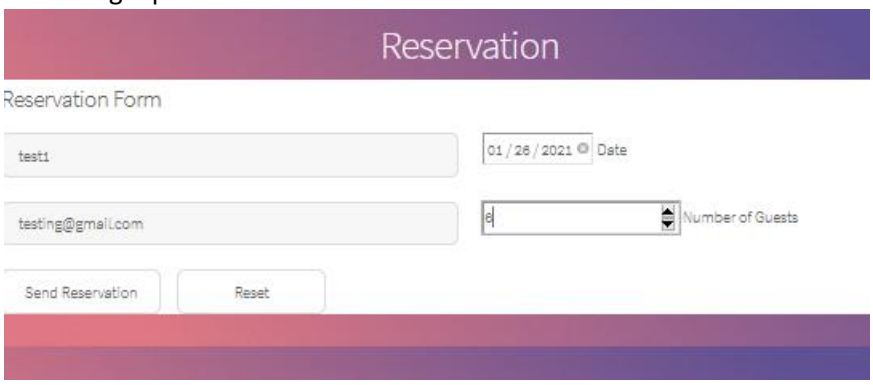
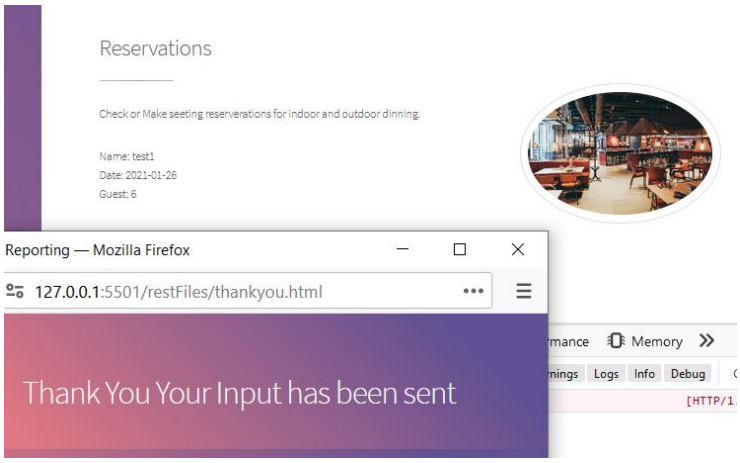
-
- Updating the cart it will show the next added bulk items



-
- Now testing added more items. Randomly adding featured item 3 and bulk items 1 and 6. For testing cart should reflect the quantity of these items increment by one.



- Reservation Creation
  - Testing reservation button is working
    - 

  - Information being inputed
    - 

- Once made the thank you window will open and reservation is saved on main page
-
  -